🔐 Spring Security Mastery Guide

From Fundamentals to Production-Grade Security

JWT • OAuth2 • RBAC • Microservices • Interview Prep

Spring Boot 3.x | Spring Security 6.x | Java 17+

📋 Table of Contents

#	Section	Page
1	Introduction to Security	3
2	Spring Security Fundamentals	6
3	Authentication Mechanisms	10
4	JWT Deep Dive	14
5	Refresh Token Strategy	21
6	Role-Based Access Control (RBAC)	25
7	OAuth 2.0 Deep Dive	30
8	OAuth2 with Spring Boot	37
9	Authorization Server & Resource Server	43
10	Securing Microservices	49
11	Security Best Practices	55
12	Logging & Auditing	60
13	Deployment Considerations	64
14	Testing Security	68
15	Real-World Project Walkthrough	74
16	Interview Preparation (50+ Questions)	85
17	Common Mistakes & Anti-Patterns	100
18	How to Think About Security Design	105

Section 01 Introduction to Security

🔐 Introduction to Security

Authentication vs Authorization

Concept	Authentication vs Authorization
Definition	Verifying WHO you are vs Determining WHAT you can do
Question	Are you really John? vs Can John access /admin?
Mechanism	Password, Biometric, OTP vs Roles, Permissions, Policies
Spring Class	AuthenticationManager vs AccessDecisionManager
HTTP Status	401 Unauthorized vs 403 Forbidden
Order	Happens FIRST vs Happens AFTER authentication

Aspect	Authentication	Authorization
Concept	Authentication	Authorization
Definition	Verifying WHO you are	Determining WHAT you can do
Question	Are you really John?	Can John access /admin?
Mechanism	Password, Biometric, OTP	Roles, Permissions, Policies
Spring Class	AuthenticationManager	AccessDecisionManager
HTTP Status on Fail	401 Unauthorized	403 Forbidden

Why Security is Critical

In 2023, the average cost of a data breach reached $4.45 million (IBM). For product-based companies, security isn't optional — it's foundational. Here's why:

- Data breaches destroy customer trust and company reputation instantly
- Regulatory compliance (GDPR, HIPAA, SOC2) requires robust security controls
- Financial fraud, IP theft, and ransomware are direct business risks
- Interview panels at top companies deeply assess security mindset
- Poorly secured APIs are the #1 attack vector in modern web applications

OWASP Top 10 — What Every Developer Must Know

The Open Web Application Security Project (OWASP) maintains the definitive list of critical web security risks:

#	Vulnerability	Example Attack	Spring Defense
A01	Broken Access Control	Accessing /admin without role	Spring Security RBAC + @PreAuthorize
A02	Cryptographic Failures	Storing passwords in plain text	BCryptPasswordEncoder
A03	Injection (SQLi, NoSQLi)	' OR '1'='1 in login form	Parameterized queries, JPA
A04	Insecure Design	No rate limiting on login	Rate limiter + account lockout
A05	Security Misconfiguration	Default credentials left unchanged	Strict SecurityFilterChain config
A06	Vulnerable Components	Outdated Spring Security jar	Dependency scanning (Snyk/OWASP)
A07	Auth & Session Failures	Session fixation, JWT in localStorage	Stateless JWT + HttpOnly cookies
A08	Software Integrity Failures	Malicious CI/CD pipeline	Secrets vault, signed artifacts
A09	Logging Failures	No audit trail for admin actions	AuditEventRepository + MDC logging
A10	SSRF	Fetching internal URLs via user input	Validate & allowlist URLs

⚡ Quick Revision Notes

- Authentication = WHO you are (401 on failure), Authorization = WHAT you can do (403 on failure)
- Spring Security processes Authentication BEFORE Authorization in the filter chain
- OWASP Top 10 is must-know for any security interview — memorize A01-A10 and their Spring defenses
- BCrypt is the standard for password hashing — never use MD5/SHA1 for passwords
- Stateless JWT-based auth is preferred for microservices; sessions for monoliths

💡 **Pro Tip:** In interviews, always mention OWASP when asked about security. Saying 'I follow OWASP Top 10 guidelines' instantly signals senior-level thinking.

Section 02 Spring Security Fundamentals

🧱 Spring Security Fundamentals

Architecture Overview

Spring Security is a powerful, highly customizable authentication and access-control framework. It is the de-facto standard for securing Spring-based applications.

> 🏗️ **Core Architecture Components**
> SecurityFilterChain → AuthenticationManager → AuthenticationProvider → UserDetailsService → SecurityContext

The Security Filter Chain

Every HTTP request passes through a chain of filters before reaching your controller. These filters implement cross-cutting security concerns:

Filter Name	Order	Purpose
`WebAsyncManagerIntegrationFilter`	100	Integrates SecurityContext with async requests
`SecurityContextPersistenceFilter`	200	Loads/saves SecurityContext between requests
`HeaderWriterFilter`	300	Writes security headers (X-Frame-Options, etc.)
`CsrfFilter`	400	CSRF token validation
`LogoutFilter`	500	Handles logout requests
`UsernamePasswordAuthenticationFilter`	600	Processes form-based login
`BasicAuthenticationFilter`	700	Processes HTTP Basic Auth headers
`BearerTokenAuthenticationFilter`	700	Processes JWT Bearer tokens (OAuth2)
`RequestCacheAwareFilter`	800	Restores cached requests after auth
`SecurityContextHolderAwareRequestFilter`	900	Wraps request with security methods
`AnonymousAuthenticationFilter`	1000	Sets anonymous auth if none present
`ExceptionTranslationFilter`	1100	Converts security exceptions to HTTP responses
`FilterSecurityInterceptor`	1200	Final authorization decision gate

Core Components Deep Dive

1. SecurityContextHolder

The heart of Spring Security. Stores the security context of the currently-executing principal.

```
// Get current authenticated user anywhere in code
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> roles = auth.getAuthorities();

// Store custom authentication (e.g., after JWT validation)
UsernamePasswordAuthenticationToken authToken =
    new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
SecurityContextHolder.getContext().setAuthentication(authToken);
```

2. AuthenticationManager & Provider

```
// AuthenticationManager delegates to AuthenticationProviders
@Bean
public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}

// DaoAuthenticationProvider is the default for username/password auth
@Bean
public DaoAuthenticationProvider authProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService); // loads user from DB
    provider.setPasswordEncoder(passwordEncoder());      // BCrypt
    return provider;
}
```

3. UserDetailsService

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {
        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found: " + username));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getEmail())
            .password(user.getPassword())
            .authorities(user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority(role.getName()))
                .collect(Collectors.toList()))
            .accountExpired(!user.isActive())
            .credentialsExpired(false)
            .disabled(!user.isEnabled())
```

```
            .build();
    }
}
```

4. SecurityFilterChain Configuration (Spring Boot 3.x)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthFilter;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())           // disable for REST APIs
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(unauthorizedHandler)
                .accessDeniedHandler(accessDeniedHandler))
            .build();
    }
}
```

How Spring Security Processes a Request

Understanding the request lifecycle is crucial for debugging and customization:

📊 Request Flow (ASCII Diagram)

HTTP Request ↓ [DelegatingFilterProxy] — Bridges Servlet world to Spring's ApplicationContext ↓ [FilterChainProxy] — Holds multiple SecurityFilterChain instances ↓ [SecurityFilterChain] — Selects matching chain based on URL pattern ↓ Filter 1 → Filter 2 → ... → Filter N (ordered chain) ↓ [ExceptionTranslationFilter] — Catches AuthenticationException / AccessDeniedException ↓ [FilterSecurityInterceptor] — Enforces authorization rules ↓ DispatcherServlet → Controller → Response

⚡ Quick Revision Notes

- SecurityContextHolder uses ThreadLocal by default — cleared after each request

- AuthenticationManager.authenticate() returns Authentication or throws AuthenticationException
- UserDetailsService.loadUserByUsername() NEVER returns null — throw UsernameNotFoundException
- Spring Boot 3.x removed WebSecurityConfigurerAdapter — use SecurityFilterChain @Bean instead
- STATELESS session policy means NO HttpSession — token must be sent with every request
- Order of filters matters: JWT filter BEFORE UsernamePasswordAuthenticationFilter

💡 **Pro Tip:** Never extend WebSecurityConfigurerAdapter in Spring Boot 3.x — it's removed. Always use the new SecurityFilterChain bean approach. This is a common interview pitfall.

Section 03 Authentication Mechanisms

🔑 Authentication Mechanisms

Basic Authentication

HTTP Basic Auth sends credentials Base64-encoded in the Authorization header. Simple but has limitations:

⚠️ **Basic Auth Flow**

Client → Base64(username:password) in header → Server decodes → validates → responds Header format: Authorization: Basic dXNlcjpwYXNzd29yZA==

```
// Enable Basic Auth (Spring Boot 3.x)
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .httpBasic(basic -> basic
            .realmName("My App")
            .authenticationEntryPoint(customEntryPoint()))
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated())
        .build();
}

// Test with curl:
// curl -u username:password http://localhost:8080/api/protected
```

Aspect	Details
Pros	Simple to implement, Works with any HTTP client, No token management
Cons	Credentials sent every request, No logout mechanism, Must use HTTPS always
Use Case	Internal service-to-service calls, Development/testing, Simple internal tools
Never Use For	Public APIs, User-facing applications, Mobile apps

Form-Based Authentication

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .formLogin(form -> form
            .loginPage("/login")                    // Custom login page URL
            .loginProcessingUrl("/api/auth/login")  // POST endpoint for
credentials
            .defaultSuccessUrl("/dashboard", true)  // Redirect after success
            .failureUrl("/login?error=true")        // Redirect after failure
            .usernameParameter("email")             // Custom field names
            .passwordParameter("pwd")
```

```
            .successHandler(customSuccessHandler()) // Custom success handling
            .failureHandler(customFailureHandler()))
        .logout(logout -> logout
            .logoutUrl("/api/auth/logout")
            .logoutSuccessUrl("/login?logout=true")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID"))
        .build();
}
```

Stateless Authentication (The Modern Standard)

For REST APIs and microservices, stateless authentication (using JWT) is the industry standard. No server-side session storage is needed.

Aspect	Session-Based	Token-Based (JWT)
Aspect	Session-Based	Token-Based (JWT)
State	Stateful (server stores session)	Stateless (client holds token)
Scalability	Sticky sessions or shared cache	Any instance can validate
Storage	Server memory/Redis	Client (memory/cookie)
Logout	Delete server session	Token blacklist or expiry
Microservices	Complex cross-service sessions	Token propagation is simple

⚡ Quick Revision Notes

- Basic Auth = Base64(user:pass) in header — NOT encryption, easily decoded
- Form login uses JSESSIONID cookie by default — stateful, not suitable for APIs
- Stateless (JWT) = no server-side session — scales horizontally with no shared state
- Always use HTTPS with Basic Auth — Base64 is NOT secure over HTTP
- For REST APIs, always use stateless auth with STATELESS session creation policy

Section 04 JWT Deep Dive

🎟 JWT (JSON Web Token) Deep Dive

What is JWT and Why Use It?

JWT (RFC 7519) is a compact, URL-safe way to represent claims between two parties. It enables stateless authentication across distributed systems.

JWT Structure

🔑 **JWT = Header.Payload.Signature**

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ← Header (Base64url) .eyJzdWIiOiJ1c2VyMTIzIiwicm9sZXMiOlsiUk9MRV9VU0VSIl0sImlhdCI6MTcwMDAwMDAwMCwiZXhwIjoxNzAwMDAzNjAwfQ== ← Payload .SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c ← Signature

Part	Content	Example
Part	Content	Example
Header	Algorithm + token type (Base64url encoded)	{"alg":"HS256","typ":"JWT"}
Payload	Claims: sub, iat, exp, roles, custom (Base64url)	{"sub":"user123","roles": ["ADMIN"],"exp":1700003600}
Signature	HMACSHA256(header + payload, secret)	Prevents tampering — server verifies this

Standard JWT Claims

Claim	Description
sub	Subject — typically the user ID or email
iat	Issued At — Unix timestamp when token was created
exp	Expiration Time — Unix timestamp when token expires (ALWAYS include this)
nbf	Not Before — token invalid before this time
jti	JWT ID — unique identifier for the token (useful for revocation)
iss	Issuer — who issued the token (your auth server URL)
aud	Audience — intended recipient of the token

Complete JWT Implementation

Dependencies (pom.xml)

```
<dependencies>
    <!-- Spring Boot Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- JJWT - Industry standard JWT library -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.3</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.3</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.3</version>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

JwtTokenProvider — Core Token Logic

```
@Component
public class JwtTokenProvider {

    // NEVER hardcode secret — use environment variables or Vault
    @Value("${app.jwt.secret}")
    private String jwtSecret;

    @Value("${app.jwt.access-token-expiration}")
    private long accessTokenExpiry; // e.g., 900000 (15 minutes)

    @Value("${app.jwt.refresh-token-expiration}")
    private long refreshTokenExpiry; // e.g., 604800000 (7 days)

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(jwtSecret);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String generateAccessToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("roles", userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.toList()));

        return buildToken(claims, userDetails.getUsername(), accessTokenExpiry);
    }
```

```
    public String generateRefreshToken(UserDetails userDetails) {
        return buildToken(new HashMap<>(), userDetails.getUsername(),
refreshTokenExpiry);
    }

    private String buildToken(Map<String, Object> extraClaims,
                              String subject, long expiration) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(subject)
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + expiration))
            .id(UUID.randomUUID().toString()) // jti claim for revocation
            .signWith(getSigningKey())
            .compact();
    }

    public Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    public String extractUsername(String token) {
        return extractAllClaims(token).getSubject();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !
isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractAllClaims(token).getExpiration().before(new Date());
    }
}
```

JwtAuthenticationFilter — Intercepts Every Request

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider jwtProvider;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        // Skip if no Bearer token present
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
```

```
            return;
        }

        final String jwt = authHeader.substring(7); // Remove "Bearer " prefix

        try {
            final String username = jwtProvider.extractUsername(jwt);

            // Only authenticate if not already authenticated
            if (username != null &&
                SecurityContextHolder.getContext().getAuthentication() == null) {

                UserDetails userDetails = userDetailsService
                    .loadUserByUsername(username);

                if (jwtProvider.isTokenValid(jwt, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails, null, userDetails.getAuthorities());
                    authToken.setDetails(
                        new
WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext()
                        .setAuthentication(authToken);
                }
            }
        } catch (ExpiredJwtException e) {
            // Token expired — client should use refresh token
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.getWriter().write("Token expired");
            return;
        } catch (JwtException e) {
            // Invalid token — possible tampering
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.getWriter().write("Invalid token");
            return;
        }

        filterChain.doFilter(request, response);
    }
}
```

Auth Controller — Login & Token Response

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtTokenProvider jwtProvider;
    private final UserDetailsService userDetailsService;
    private final RefreshTokenService refreshTokenService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest
request) {
        // This throws AuthenticationException if credentials are invalid
        Authentication authentication = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
```

```
                request.getEmail(), request.getPassword()));

        UserDetails userDetails = (UserDetails) authentication.getPrincipal();

        String accessToken = jwtProvider.generateAccessToken(userDetails);
        String refreshToken =
refreshTokenService.createRefreshToken(userDetails.getUsername());

        return ResponseEntity.ok(AuthResponse.builder()
            .accessToken(accessToken)
            .refreshToken(refreshToken)
            .tokenType("Bearer")
            .expiresIn(900) // 15 minutes in seconds
            .build());
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refreshToken(@RequestBody RefreshRequest
request) {
        return refreshTokenService.findByToken(request.getRefreshToken())
            .map(refreshTokenService::verifyExpiration)
            .map(RefreshToken::getUser)
            .map(user -> {
                UserDetails userDetails = userDetailsService
                    .loadUserByUsername(user.getEmail());
                String newAccessToken =
jwtProvider.generateAccessToken(userDetails);
                return ResponseEntity.ok(AuthResponse.builder()
                    .accessToken(newAccessToken)
                    .tokenType("Bearer")
                    .expiresIn(900)
                    .build());
            })
            .orElseThrow(() -> new RefreshTokenException("Invalid refresh
token"));
    }
}
// Curl Example:
// curl -X POST http://localhost:8080/api/auth/login \
//   -H "Content-Type: application/json" \
//   -d '{"email":"user@example.com","password":"SecurePass123!"}'
```

Secure Secret Key Generation

```
// NEVER use a weak secret! Generate with:
// openssl rand -base64 64

// application.yml
app:
  jwt:
    # 512-bit secret (64 bytes, base64 encoded) for HS512
    secret: 7A9B2C4D6E8F1A3B5C7D9E0F2A4B6C8D0E2F4A6B8C0D2E4F6A8B0C2D4E6F8A0
    access-token-expiration: 900000      # 15 minutes
    refresh-token-expiration: 604800000  # 7 days
```

Access Token vs Refresh Token

Property	Access Token	Refresh Token

Property	Access Token	Refresh Token
Purpose	Authenticate API calls	Get new access tokens
Lifetime	Short (5-15 minutes)	Long (7-30 days)
Storage	Memory (JS) or HttpOnly cookie	HttpOnly cookie or secure DB
Sent With	Every API request header	Only to /auth/refresh endpoint
Revocable	Hard (would need blacklist)	Yes (delete from DB)
Rotation	Issued frequently	Rotate on every use (best practice)

⚡ Quick Revision Notes

- JWT = Header.Payload.Signature — all Base64url encoded, NOT encrypted by default
- Always set exp claim — JWT without expiry is a security vulnerability
- Use jti claim for token revocation — store revoked jti values in Redis with TTL
- HS256 uses shared secret, RS256 uses private/public key pair (better for microservices)
- Access token: 15 min lifetime | Refresh token: 7 days, stored in DB/Redis
- NEVER put sensitive data (passwords, SSNs) in JWT payload — it's Base64, not encrypted

💡 **Pro Tip:** Store access tokens in memory (JavaScript variable) and refresh tokens in HttpOnly cookies. This prevents XSS attacks from stealing tokens. NEVER store JWTs in localStorage.

Section 05 Refresh Token Strategy

🔄 Refresh Token Strategy

Why Refresh Tokens are Needed

Short-lived access tokens minimize damage if compromised, but force users to re-login frequently. Refresh tokens solve this by silently issuing new access tokens without re-authentication.

Database Storage Strategy

```
// RefreshToken Entity
@Entity
@Table(name = "refresh_tokens")
public class RefreshToken {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @OneToOne
    @JoinColumn(name = "user_id", referencedColumnName = "id")
    private User user;

    @Column(nullable = false, unique = true)
    private String token; // hashed token, not plain

    @Column(nullable = false)
    private Instant expiryDate;

    private boolean revoked = false;

    // Device info for multi-device support
    private String deviceId;
    private String userAgent;
    private String ipAddress;
}

// RefreshTokenService
@Service
@Transactional
public class RefreshTokenService {
    @Value("${app.jwt.refresh-token-expiration}")
    private Long refreshTokenDurationMs;

    @Autowired private RefreshTokenRepository refreshTokenRepository;
    @Autowired private UserRepository userRepository;

    public String createRefreshToken(String username) {
        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new UserNotFoundException(username));

        // Token rotation: delete old tokens for this user+device
        refreshTokenRepository.deleteByUserAndDeviceId(user,
getCurrentDeviceId());
```

```
        String rawToken = UUID.randomUUID().toString();
        String hashedToken = hashToken(rawToken); // Store hash, not plain token

        RefreshToken refreshToken = new RefreshToken();
        refreshToken.setUser(user);
        refreshToken.setToken(hashedToken);

refreshToken.setExpiryDate(Instant.now().plusMillis(refreshTokenDurationMs));
        refreshTokenRepository.save(refreshToken);

        return rawToken; // Return plain token to client
    }

    public RefreshToken verifyExpiration(RefreshToken token) {
        if (token.getExpiryDate().compareTo(Instant.now()) < 0) {
            refreshTokenRepository.delete(token);
            throw new RefreshTokenException("Refresh token expired. Please re-
login");
        }
        return token;
    }

    private String hashToken(String token) {
        return DigestUtils.sha256Hex(token); // Apache Commons Codec
    }
}
```

Redis Storage Strategy (Production Preferred)

```
// Redis-based refresh token storage
@Service
public class RedisRefreshTokenService {

    @Autowired
    private RedisTemplate<String, String> redisTemplate;

    private static final String REFRESH_TOKEN_PREFIX = "refresh_token:";
    private static final String USER_TOKENS_PREFIX = "user_tokens:";

    public String createRefreshToken(String userId) {
        String token = UUID.randomUUID().toString();
        String key = REFRESH_TOKEN_PREFIX + token;

        // Store token with 7 day TTL
        redisTemplate.opsForValue().set(key, userId, 7, TimeUnit.DAYS);

        // Track all tokens for user (for logout all devices)
        redisTemplate.opsForSet().add(USER_TOKENS_PREFIX + userId, token);

        return token;
    }

    public Optional<String> getUserIdByToken(String token) {
        String key = REFRESH_TOKEN_PREFIX + token;
        String userId = redisTemplate.opsForValue().get(key);
        return Optional.ofNullable(userId);
    }

    public void revokeToken(String token) {
```

```
        String userId = redisTemplate.opsForValue().get(REFRESH_TOKEN_PREFIX +
token);
        redisTemplate.delete(REFRESH_TOKEN_PREFIX + token);
        if (userId != null) {
            redisTemplate.opsForSet().remove(USER_TOKENS_PREFIX + userId, token);
        }
    }

    public void revokeAllUserTokens(String userId) {
        // Logout from all devices
        Set<String> tokens = redisTemplate.opsForSet()
            .members(USER_TOKENS_PREFIX + userId);
        if (tokens != null) {
            tokens.forEach(token ->
                redisTemplate.delete(REFRESH_TOKEN_PREFIX + token));
            redisTemplate.delete(USER_TOKENS_PREFIX + userId);
        }
    }
}
```

Token Rotation Strategy

🔄 **Refresh Token Rotation**

Best Practice: Issue a NEW refresh token every time the old one is used. Old refresh token is immediately invalidated. If an attacker steals an old refresh token and tries to use it → detected as reuse → revoke all tokens for that user.

Strategy	Description
Simple Rotation	New refresh token on every use, old one deleted — prevents replay attacks
Reuse Detection	If old token used again = someone stole it → revoke ALL user sessions
Family Tracking	Track token 'family' — any reuse in same family revokes entire family
Sliding Window	Extend expiry on every use (max cap still applies)

⚡ **Quick Revision Notes**

- Never store plain refresh tokens in DB — store SHA-256 hash
- Redis is preferred for refresh tokens in microservices (TTL, fast lookups, cross-service access)
- Token rotation: rotate on every use, detect and alert on reuse
- Support 'logout from all devices' by tracking all tokens per user
- Use device fingerprinting (user-agent + device ID) for multi-device token management

💡 **Pro Tip:** Implement refresh token reuse detection. When a refresh token that was already used is presented again, it means a token was stolen. Immediately revoke ALL tokens for that user and force re-login.

Section 06 Role-Based Access Control

🛡️ Role-Based Access Control (RBAC)

Roles vs Authorities (Permissions)

Concept	Role	Authority
Concept	Role	Authority (Permission)
Prefix	ROLE_ prefix required	No prefix restriction
Granularity	Coarse-grained (ADMIN, USER)	Fine-grained (user:read, order:write)
Spring Check	hasRole('ADMIN') [adds ROLE_ automatically]	hasAuthority('user:create')
Example	ROLE_ADMIN, ROLE_USER	product:delete, report:view
Assignment	One or few per user	Many possible per user

```
// Entity Design: User → Roles → Permissions
@Entity
public class User {
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles")
    private Set<Role> roles;
}

@Entity
public class Role {
    private String name; // "ROLE_ADMIN", "ROLE_USER"

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "role_permissions")
    private Set<Permission> permissions;
}

@Entity
public class Permission {
    private String name; // "user:read", "user:write", "product:delete"
}

// UserDetailsService — flatten roles + permissions into authorities
@Override
public UserDetails loadUserByUsername(String username) {
    User user = userRepository.findByEmail(username).orElseThrow(...);

    List<GrantedAuthority> authorities = user.getRoles().stream()
        .flatMap(role -> {
            List<GrantedAuthority> perms = new ArrayList<>();
            // Add the role itself
            perms.add(new SimpleGrantedAuthority(role.getName()));
            // Add all permissions under this role
            role.getPermissions().forEach(perm ->
                perms.add(new SimpleGrantedAuthority(perm.getName())));
```

```
            return perms.stream();
        })
        .collect(Collectors.toList());

    return new org.springframework.security.core.userdetails.User(
        user.getEmail(), user.getPassword(), authorities);
}
```

Method-Level Security

@PreAuthorize — Before Method Executes

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    // Only ADMIN can see all users
    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public List<UserDTO> getAllUsers() { ... }

    // User can access own profile; ADMIN can access any
    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
    public UserDTO getUser(@PathVariable Long id) { ... }

    // SpEL with custom permission evaluator
    @DeleteMapping("/{id}")
    @PreAuthorize("hasAuthority('user:delete')")
    public void deleteUser(@PathVariable Long id) { ... }

    // Method argument validation
    @PostMapping
    @PreAuthorize("hasRole('ADMIN') and #user.role != 'SUPER_ADMIN'")
    public UserDTO createUser(@RequestBody CreateUserRequest user) { ... }

    // Service layer security
    @PostAuthorize("returnObject.ownerId == authentication.principal.id")
    public Order getOrderDetails(Long orderId) { ... }
}
```

URL-Based Authorization (SecurityFilterChain)

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            // Public endpoints — no auth required
            .requestMatchers("/api/auth/**", "/api/public/**",
                            "/swagger-ui/**", "/actuator/health").permitAll()

            // Admin-only endpoints
            .requestMatchers("/api/admin/**").hasRole("ADMIN")

            // Specific HTTP methods
            .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
            .requestMatchers(HttpMethod.POST,
"/api/products/**").hasAuthority("product:create")
            .requestMatchers(HttpMethod.DELETE,
```

```
"/api/products/**").hasAuthority("product:delete")

            // Multiple roles
            .requestMatchers("/api/reports/**").hasAnyRole("ADMIN", "MANAGER")

            // All other requests need authentication
            .anyRequest().authenticated())
        .build();
}
```

Hierarchical Roles

```
// Role hierarchy: ADMIN > MANAGER > USER
@Bean
public RoleHierarchy roleHierarchy() {
    RoleHierarchyImpl hierarchy = new RoleHierarchyImpl();
    hierarchy.setHierarchy("ROLE_ADMIN > ROLE_MANAGER > ROLE_USER");
    return hierarchy;
}

@Bean
public DefaultWebSecurityExpressionHandler expressionHandler() {
    DefaultWebSecurityExpressionHandler handler =
        new DefaultWebSecurityExpressionHandler();
    handler.setRoleHierarchy(roleHierarchy());
    return handler;
}
// Now ADMIN automatically has MANAGER and USER permissions too
```

⚡ Quick Revision Notes

- hasRole('ADMIN') adds ROLE_ prefix automatically; hasAuthority('ROLE_ADMIN') does not
- @PreAuthorize uses SpEL — can access method arguments with #paramName
- @PostAuthorize checks AFTER method executes — useful for filtering response data
- Enable method security with @EnableMethodSecurity(prePostEnabled = true)
- Role hierarchy means ADMIN inherits all permissions of lower roles
- Always prefer fine-grained permissions over coarse roles for enterprise apps

💡 **Pro Tip:** Design your permission system around actions (user:read, order:cancel) rather than roles alone. This gives much more flexibility — a 'MANAGER' might need some ADMIN permissions without full admin access.

Section 07 OAuth 2.0 Deep Dive

🌐 OAuth 2.0 Deep Dive

What is OAuth 2.0?

OAuth 2.0 is an authorization framework (RFC 6749) that enables applications to obtain limited access to user accounts on an HTTP service without sharing credentials. Think of it as a 'valet key' for your digital accounts.

OAuth 2.0 Roles

Role	Description
Resource Owner	The user (e.g., John who has a Google account)
Client	Your application requesting access (e.g., your Spring Boot app)
Authorization Server	Issues tokens after authenticating the user (e.g., Google's OAuth server)
Resource Server	Holds the protected resources (e.g., Google's API with user's profile data)

Grant Types — When to Use Which

1. Authorization Code Flow (Most Secure — Use This for Web Apps)

📊 **Authorization Code Flow**

User → Click 'Login with Google' App → Redirect to Google: /authorize?client_id=xxx&redirect_uri=xxx&scope=openid email&state=random123 Google → User authenticates, consents Google → Redirect back: /callback?code=AUTH_CODE&state=random123 App → Exchange code for token (server-to-server): POST /token {code, client_secret} Google → Returns: {access_token, refresh_token, id_token} App → Use access_token to call Google APIs

2. Authorization Code + PKCE (For SPAs and Mobile)

```
// PKCE (Proof Key for Code Exchange) prevents auth code interception
// Step 1: Generate code verifier (random 43-128 char string)
String codeVerifier = generateSecureRandom(64); // store this securely

// Step 2: Create code challenge = BASE64URL(SHA256(codeVerifier))
String codeChallenge = base64UrlEncode(sha256(codeVerifier));

// Step 3: Include in authorization request
GET /authorize?
    client_id=xxx&
    response_type=code&
    code_challenge=<codeChallenge>&
    code_challenge_method=S256&
    redirect_uri=xxx

// Step 4: Exchange code (send verifier, NOT challenge)
```

```
POST /token
    code=AUTH_CODE&
    code_verifier=<codeVerifier>   // Server verifies SHA256(verifier) ==
challenge
```

3. Client Credentials (Service-to-Service)

```
// No user involved — machine-to-machine auth
POST /oauth2/token
Authorization: Basic <base64(clientId:clientSecret)>
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=api.read

// Returns:
{
    "access_token": "eyJ...",
    "token_type": "Bearer",
    "expires_in": 3600,
    "scope": "api.read"
}

// Spring RestClient usage
@Bean
public RestClient oauth2RestClient(OAuth2AuthorizedClientManager manager) {
    return RestClient.builder()
        .baseUrl("https://api.partner-service.com")
        .requestInterceptor(new OAuth2ClientHttpRequestInterceptor(manager))
        .build();
}
```

4. Password Grant (Legacy — Avoid in New Systems)

> ⚠️ **Password Grant is Deprecated**
> Client collects username/password directly and sends to auth server. This violates the core OAuth2 principle of not sharing credentials with clients. Use only for trusted first-party apps during migration. NEVER for third-party apps.

OAuth2 Scopes — Fine-Grained Authorization

Scope	What It Allows
openid	OpenID Connect — get user identity (sub claim)
profile	User's basic profile (name, picture)
email	User's email address
read:data	Read-only access to data
write:data	Write access to data
offline_access	Get refresh token for offline access

⚡ Quick Revision Notes

- OAuth2 is for AUTHORIZATION not authentication — OpenID Connect (OIDC) adds identity layer
- Authorization Code Flow: frontend gets code, backend exchanges for token (code never stored)
- Always use PKCE for public clients (SPAs, mobile apps) — prevents code interception
- Client Credentials: no user, just app-to-app auth with client ID + secret
- state parameter prevents CSRF attacks in OAuth2 flow — always validate it
- Never use Password Grant for third-party apps — credential exposure risk

💡 **Pro Tip:** OIDC = OAuth2 + ID Token. The ID token (a JWT) contains user identity info. Use it for 'Login with Google/GitHub'. The access token is for calling APIs on behalf of the user. These are DIFFERENT things — confusing them is a common interview mistake.

Section 08 OAuth2 with Spring Boot

🔗 OAuth2 with Spring Boot

Dependencies

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

application.yml — Google & GitHub OAuth2

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: openid, profile, email
            redirect-uri: "${APP_BASE_URL}/login/oauth2/code/google"
          github:
            client-id: ${GITHUB_CLIENT_ID}
            client-secret: ${GITHUB_CLIENT_SECRET}
            scope: user:email, read:user
        provider:
          google:
            authorization-uri: https://accounts.google.com/o/oauth2/v2/auth
            token-uri: https://oauth2.googleapis.com/token
            user-info-uri: https://www.googleapis.com/oauth2/v3/userinfo
            user-name-attribute: sub
```

Security Configuration for OAuth2

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .oauth2Login(oauth2 -> oauth2
            .loginPage("/login")
            .userInfoEndpoint(userInfo -> userInfo
                .userService(customOAuth2UserService()))  // Map OAuth user to our
domain
            .successHandler(oAuth2AuthSuccessHandler())   // Generate JWT after
OAuth login
            .failureHandler(oAuth2AuthFailureHandler()))  // Handle failures
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/login", "/error").permitAll()
            .anyRequest().authenticated())
        .build();
}
```

Custom OAuth2 User Service — Map to Internal User

```
@Service
public class CustomOAuth2UserService extends DefaultOAuth2UserService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public OAuth2User loadUser(OAuth2UserRequest request) throws
OAuth2AuthenticationException {
        OAuth2User oauth2User = super.loadUser(request);

        String provider = request.getClientRegistration().getRegistrationId(); //
"google"/"github"
        String providerId = oauth2User.getName(); // 'sub' claim for Google
        String email = oauth2User.getAttribute("email");
        String name = oauth2User.getAttribute("name");
        String picture = oauth2User.getAttribute("picture");

        // Find or create internal user
        User user = userRepository.findByEmail(email)
            .orElseGet(() -> createNewOAuthUser(email, name, picture, provider,
providerId));

        // Update provider info if first OAuth login
        if (user.getProvider() == null) {
            user.setProvider(provider);
            user.setProviderId(providerId);
            userRepository.save(user);
        }

        return UserPrincipal.create(user, oauth2User.getAttributes());
    }

    private User createNewOAuthUser(String email, String name,
                                    String picture, String provider, String
providerId) {
        User user = new User();
        user.setEmail(email);
        user.setName(name);
        user.setProfilePicture(picture);
        user.setProvider(provider);
        user.setProviderId(providerId);
        user.setPassword(null); // OAuth users don't have passwords
        user.setRoles(Set.of(defaultUserRole()));
        user.setEmailVerified(true); // Email verified by OAuth provider
        return userRepository.save(user);
    }
}
```

OAuth2 Success Handler — Issue JWT

```
@Component
public class OAuth2AuthSuccessHandler extends
SimpleUrlAuthenticationSuccessHandler {

    @Autowired private JwtTokenProvider jwtProvider;
    @Autowired private RefreshTokenService refreshTokenService;
```

```
    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                        HttpServletResponse response,
                                        Authentication authentication) throws
IOException {

        UserPrincipal userPrincipal = (UserPrincipal)
authentication.getPrincipal();

        // Generate JWT tokens
        String accessToken = jwtProvider.generateAccessToken(userPrincipal);
        String refreshToken =
refreshTokenService.createRefreshToken(userPrincipal.getEmail());

        // Set refresh token as HttpOnly cookie
        Cookie refreshCookie = new Cookie("refreshToken", refreshToken);
        refreshCookie.setHttpOnly(true);
        refreshCookie.setSecure(true); // HTTPS only
        refreshCookie.setPath("/api/auth/refresh");
        refreshCookie.setMaxAge(7 * 24 * 60 * 60); // 7 days
        response.addCookie(refreshCookie);

        // Redirect to frontend with access token
        String redirectUrl = UriComponentsBuilder
            .fromUriString(determineTargetUrl(request, response))
            .queryParam("token", accessToken)
            .build().toUriString();

        getRedirectStrategy().sendRedirect(request, response, redirectUrl);
    }
}
```

⚡ Quick Revision Notes

- Spring Boot auto-configures OAuth2 login with just application.yml properties
- CustomOAuth2UserService: find-or-create pattern links OAuth identity to internal user
- After OAuth2 success, issue your own JWT — don't use Google/GitHub's access token for your APIs
- Store refresh token in HttpOnly + Secure cookie, access token in response body/memory
- Handle both OAuth and traditional login with unified UserPrincipal implementation

Section 09 Authorization & Resource Server

🏢 Authorization Server & Resource Server

What is an Authorization Server?

An Authorization Server is a dedicated service that issues tokens. In microservices, you build your own auth server using Spring Authorization Server (spring-authorization-server).

🏗️ Spring Authorization Server Setup (Spring Boot 3.x)

The Spring Authorization Server implements OAuth 2.1 and OpenID Connect 1.0. It issues access tokens (JWT), refresh tokens, and ID tokens. Resource servers validate these tokens to protect APIs.

```
<!-- Spring Authorization Server dependency -->
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-oauth2-authorization-server</artifactId>
</dependency>

// Authorization Server Configuration
@Configuration
@Import(OAuth2AuthorizationServerConfiguration.class)
public class AuthorizationServerConfig {

    @Bean
    @Order(1)
    public SecurityFilterChain authorizationServerSecurityFilterChain(HttpSecurity
http)
            throws Exception {
        OAuth2AuthorizationServerConfiguration.applyDefaultSecurity(http);
        http.getConfigurer(OAuth2AuthorizationServerConfigurer.class)
            .oidc(Customizer.withDefaults()); // Enable OpenID Connect

        return http
            .exceptionHandling(exceptions -> exceptions
                .defaultAuthenticationEntryPointFor(
                    new LoginUrlAuthenticationEntryPoint("/login"),
                    new MediaTypeRequestMatcher(MediaType.TEXT_HTML)))
            .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))
            .build();
    }

    // Register OAuth2 clients
    @Bean
    public RegisteredClientRepository registeredClientRepository() {
        RegisteredClient webClient =
RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("web-client")
            .clientSecret("{bcrypt}" + new
BCryptPasswordEncoder().encode("secret"))
            .clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_B
ASIC)
            .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
            .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
            .redirectUri("http://localhost:3000/callback")
            .scope(OidcScopes.OPENID)
            .scope(OidcScopes.PROFILE)
```

```
            .scope("api.read")
            .tokenSettings(TokenSettings.builder()
                .accessTokenTimeToLive(Duration.ofMinutes(15))
                .refreshTokenTimeToLive(Duration.ofDays(7))
                .reuseRefreshTokens(false) // Rotate refresh tokens
                .build())
            .build();

        RegisteredClient serviceClient =
RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("order-service")
            .clientSecret("{bcrypt}" + new BCryptPasswordEncoder().encode("order-
secret"))
            .clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_B
ASIC)
            .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
            .scope("product.read")
            .build();

        return new InMemoryRegisteredClientRepository(webClient, serviceClient);
    }

    // JWT configuration with RSA key pair (asymmetric — best for microservices)
    @Bean
    public JWKSource<SecurityContext> jwkSource() {
        KeyPair keyPair = generateRsaKey();
        RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
        RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();
        RSAKey rsaKey = new RSAKey.Builder(publicKey)
            .privateKey(privateKey)
            .keyID(UUID.randomUUID().toString())
            .build();
        JWKSet jwkSet = new JWKSet(rsaKey);
        return new ImmutableJWKSet<>(jwkSet);
    }

    // IMPORTANT: In production, load key pair from Vault, not generate at startup
    private static KeyPair generateRsaKey() {
        try {
            KeyPairGenerator generator = KeyPairGenerator.getInstance("RSA");
            generator.initialize(2048);
            return generator.generateKeyPair();
        } catch (Exception ex) {
            throw new IllegalStateException(ex);
        }
    }
}
```

Resource Server Configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .oauth2ResourceServer(rs -> rs
                .jwt(jwt -> jwt
```

```
                        .jwtAuthenticationConverter(jwtAuthenticationConverter())))
                .authorizeHttpRequests(auth -> auth
                    .requestMatchers("/actuator/health").permitAll()
                    .requestMatchers(HttpMethod.GET,
"/api/**").hasAuthority("SCOPE_api.read")
                    .requestMatchers(HttpMethod.POST,
"/api/**").hasAuthority("SCOPE_api.write")
                    .anyRequest().authenticated())
                .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .build();
    }

    // Convert JWT claims to Spring Security authorities
    @Bean
    public JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtGrantedAuthoritiesConverter grantedAuthoritiesConverter =
            new JwtGrantedAuthoritiesConverter();
        grantedAuthoritiesConverter.setAuthoritiesClaimName("roles");
        grantedAuthoritiesConverter.setAuthorityPrefix("ROLE_");

        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(grantedAuthoritiesConverter);
        return converter;
    }
}

# application.yml for Resource Server
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          # For your own auth server — use public key endpoint
          jwk-set-uri: http://auth-server:9000/oauth2/jwks
          # OR for external provider:
          issuer-uri: https://accounts.google.com
```

HS256 vs RS256 — Key Choice

Aspect	HS256	RS256
Aspect	HS256 (Symmetric)	RS256 (Asymmetric)
Algorithm	HMAC-SHA256	RSA-SHA256
Key	One shared secret	Private key (sign) + Public key (verify)
Who can verify	Only parties with the secret	Anyone with the public key
Best for	Single-service, simple setups	Microservices, multiple resource servers
Key distribution	Secret must be shared securely	Public key freely distributable via JWKS endpoint

⚡ Quick Revision Notes

- RS256 is preferred for microservices — only auth server needs private key, any service can verify with public key
- JWKS endpoint (/oauth2/jwks) exposes public keys — resource servers auto-fetch and cache
- Spring Authorization Server implements OAuth 2.1 (stricter than 2.0 — no implicit grant, PKCE required)
- Scope authorities are prefixed with SCOPE_ by Spring — SCOPE_api.read, SCOPE_openid
- Store RSA key pair in Vault or KMS in production — never hardcode or generate on startup

Section 10 Securing Microservices

🔐 Securing Microservices

API Gateway Authentication Pattern

In microservices, the API Gateway is the single entry point. It validates tokens centrally, so individual services don't need to re-validate.

🏗️ **API Gateway Security Flow**

Client → [API Gateway: validates JWT] → Route to microservice Gateway adds X-User-Id, X-User-Roles headers after validation Services trust these headers (internal network only) Never expose services directly — always go through gateway

```
// Spring Cloud Gateway + Security configuration
@Configuration
public class GatewaySecurityConfig {

    @Bean
    public SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity
http) {
        return http
            .csrf(csrf -> csrf.disable())
            .authorizeExchange(auth -> auth
                .pathMatchers("/api/auth/**").permitAll()
                .pathMatchers("/api/public/**").permitAll()
                .anyExchange().authenticated())
            .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))
            .addFilterAt(jwtGatewayFilter(),
SecurityWebFiltersOrder.AUTHENTICATION)
            .build();
    }

    // Propagate user info to downstream services via headers
    @Bean
    public GlobalFilter userContextPropagationFilter() {
        return (exchange, chain) -> {
            Authentication auth = exchange.getAttribute(
                SecurityContext.class.getName() + ".ATTR_NAME");
            if (auth != null && auth.getPrincipal() instanceof Jwt jwt) {
                ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
                    .header("X-User-Id", jwt.getSubject())
                    .header("X-User-Email", jwt.getClaimAsString("email"))
                    .header("X-User-Roles", String.join(",",
                        jwt.getClaimAsStringList("roles")))
                    .build();
                return
chain.filter(exchange.mutate().request(mutatedRequest).build());
            }
            return chain.filter(exchange);
        };
    }
}
```

Token Propagation — Service-to-Service

```
// Propagate JWT from incoming request to downstream calls
@Configuration
public class FeignClientConfig {

    @Bean
    public RequestInterceptor requestInterceptor() {
        return template -> {
            // Get current request's JWT and forward it
            Authentication auth =
SecurityContextHolder.getContext().getAuthentication();
            if (auth instanceof JwtAuthenticationToken jwtAuth) {
                template.header("Authorization", "Bearer " +
jwtAuth.getToken().getTokenValue());
            }
        };
    }
}

// Service-to-service with Client Credentials (no user context)
@Configuration
public class ServiceToServiceConfig {

    @Bean
    public OAuth2AuthorizedClientManager authorizedClientManager(
            ClientRegistrationRepository clients,
            OAuth2AuthorizedClientRepository authorizedClients) {

        OAuth2AuthorizedClientProvider provider =
            OAuth2AuthorizedClientProviderBuilder.builder()
                .clientCredentials()
                .build();

        DefaultOAuth2AuthorizedClientManager manager =
            new DefaultOAuth2AuthorizedClientManager(clients, authorizedClients);
        manager.setAuthorizedClientProvider(provider);
        return manager;
    }

    @Bean
    public RestClient productServiceClient(OAuth2AuthorizedClientManager manager)
{
        return RestClient.builder()
            .baseUrl("http://product-service")
            .requestInterceptor(new OAuth2ClientHttpRequestInterceptor(manager))
            .build();
    }
}
```

Centralized Auth Service Architecture

Component	Responsibility
Auth Service	Handles login, registration, token issuance and refresh — the single source of truth
API Gateway	Validates tokens, routes requests, propagates user context headers

Microservices	Trust gateway headers, apply RBAC, never call auth service directly
Redis	Shared token blacklist, refresh token storage, rate limiting counters
Vault	Centralized secrets — JWT signing keys, DB credentials, API keys

```
// application.yml for microservices — trust gateway headers
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://auth-service:8080/oauth2/jwks

# Route configuration in API Gateway (application.yml)
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - StripPrefix=1
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100
                redis-rate-limiter.burstCapacity: 200
```

⚡ Quick Revision Notes

- API Gateway = security perimeter — validate all tokens here, propagate user context via headers
- Service-to-service: use Client Credentials OAuth2 flow (no user involved)
- User request forwarding: propagate original JWT OR extract user info into trusted headers
- Never expose microservices directly to internet — only through API Gateway
- Use service mesh (Istio) for mTLS between services in production

💡 **Pro Tip:** In interviews, draw the flow: Client → Gateway (JWT validation) → Service (trusts X-User-* headers) → Service-to-Service (Client Credentials). This shows complete microservices security understanding.

Section 11 Security Best Practices

⚙️ Security Best Practices (Production Grade)

Password Hashing — BCrypt

```
// Always use BCrypt with appropriate strength (10-12 for production)
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // 2^12 iterations
}

// Verification
passwordEncoder.matches(rawPassword, encodedPassword);

// NEVER do:
// MD5/SHA1 hashing — reversible via rainbow tables
// No hashing — catastrophic
// Custom hashing — error-prone
```

Algorithm	Notes
BCrypt (Recommended)	Adaptive hash, includes salt, slow by design — work factor configurable
Argon2 (Best)	Winner of Password Hashing Competition, use with Spring Security 5.8+
PBKDF2	NIST recommended, FIPS compliant — use for regulated industries
SHA-256/MD5	NEVER use for passwords — fast = easy to brute force

CSRF Protection

```
// CSRF is needed for browser-based form submissions
// DISABLE for stateless REST APIs (JWT-based)
// ENABLE for server-rendered apps with sessions

// For REST APIs (disable CSRF):
.csrf(csrf -> csrf.disable())

// For hybrid apps (enable with custom token repository):
.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    .csrfTokenRequestHandler(new CsrfTokenRequestAttributeHandler()))
```

CORS Configuration

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();

    // NEVER use * in production — specify exact origins
    config.setAllowedOrigins(Arrays.asList(
        "https://app.yourdomain.com",
```

```
        "https://admin.yourdomain.com"
    ));
    config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
    config.setAllowedHeaders(Arrays.asList(
        "Authorization", "Content-Type", "X-Requested-With"
    ));
    config.setExposedHeaders(Arrays.asList("X-Auth-Token"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return source;
}
```

Security Headers

```
// Spring Security auto-adds security headers. Configure them:
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .headers(headers -> headers
            .contentTypeOptions(Customizer.withDefaults())          // X-Content-
Type-Options
            .frameOptions(frame -> frame.deny())                    // X-Frame-
Options: DENY
            .xssProtection(Customizer.withDefaults())               // X-XSS-
Protection
            .referrerPolicy(referrer -> referrer
                .policy(ReferrerPolicyHeaderWriter.ReferrerPolicy.STRICT_ORIGIN_WH
EN_CROSS_ORIGIN))
            .contentSecurityPolicy(csp -> csp
                .policyDirectives("default-src 'self'; script-src 'self'; style-
src 'self'"))
            .httpStrictTransportSecurity(hsts -> hsts
                .includeSubDomains(true)
                .preload(true)
                .maxAgeInSeconds(31536000))) // 1 year
        .build();
}
```

Rate Limiting

```
// Using Bucket4j for rate limiting
@Component
public class RateLimitingFilter extends OncePerRequestFilter {

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws IOException,
ServletException {
        String ip = getClientIp(request);
        Bucket bucket = buckets.computeIfAbsent(ip, this::createNewBucket);
```

```
        if (bucket.tryConsume(1)) {
            chain.doFilter(request, response);
        } else {
            response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
            response.addHeader("X-Rate-Limit-Retry-After-Seconds", "60");
            response.getWriter().write("Rate limit exceeded. Try again in 60
seconds.");
        }
    }

    private Bucket createNewBucket(String ip) {
        return Bucket.builder()
            .addLimit(Bandwidth.classic(100, Refill.greedy(100,
Duration.ofMinutes(1))))
            .build();
    }
}
```

⚡ Quick Revision Notes

- BCrypt strength=12 is standard — increases by ~2x per increment (10=1s, 12=4s, 14=16s)
- CSRF: disable for stateless REST APIs, enable for session-based web apps
- CORS: allowedOrigins must be explicit in production — never use '*' with allowCredentials=true
- HSTS header forces HTTPS for 1 year — submit to browser preload lists for extra security
- Rate limit per IP + per user — burst allowance for legitimate traffic spikes

Section 12 Logging & Auditing

📊 Logging & Auditing

Security Event Logging

```
// Configure security event listeners
@Component
public class SecurityEventListener {

    private static final Logger log =
LoggerFactory.getLogger(SecurityEventListener.class);

    @EventListener
    public void onAuthenticationSuccess(AuthenticationSuccessEvent event) {
        String username = event.getAuthentication().getName();
        log.info("AUTH_SUCCESS user={} ip={} timestamp={}",
            username, getClientIp(), Instant.now());
    }

    @EventListener
    public void onAuthenticationFailure(AbstractAuthenticationFailureEvent event)
{
        String username = event.getAuthentication().getName();
        String reason = event.getException().getClass().getSimpleName();
        log.warn("AUTH_FAILURE user={} reason={} ip={} timestamp={}",
            username, reason, getClientIp(), Instant.now());

        // Alert if too many failures
        if (failureCountExceeds(username, 5)) {
            alertSecurityTeam("Brute force detected for: " + username);
            lockAccount(username);
        }
    }

    @EventListener
    public void onAccessDenied(AuthorizationDeniedEvent event) {
        log.warn("ACCESS_DENIED user={} resource={} ip={}",
            getUsername(), getResource(event), getClientIp());
    }
}

// Use MDC for tracing across services
@Component
public class MdcFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        try {
            MDC.put("requestId", UUID.randomUUID().toString());
            MDC.put("userId", getCurrentUserId());
            MDC.put("ip", getClientIp(req));
            chain.doFilter(req, res);
        } finally {
            MDC.clear(); // IMPORTANT: clear MDC after each request
        }
    }
```

```
}
```

⚡ Quick Revision Notes

- Log AUTH_SUCCESS, AUTH_FAILURE, ACCESS_DENIED, LOGOUT events as minimum
- Use MDC (Mapped Diagnostic Context) for correlating logs across a request lifecycle
- Never log passwords, tokens, or PII — log user IDs instead
- Detect brute force: 5+ failures in 1 minute → lock account + alert security team
- Ship logs to centralized SIEM (Splunk, ELK) for real-time security monitoring

Section 13 Deployment Considerations

🐳 Deployment Considerations

Secrets Management

```
# WRONG: Hardcoded secrets
spring:
  datasource:
    password: mypassword123  # NEVER DO THIS

# CORRECT: Environment variables
spring:
  datasource:
    password: ${DB_PASSWORD}  # Set in CI/CD, Kubernetes secrets, or Vault

# BEST: HashiCorp Vault integration
spring:
  cloud:
    vault:
      host: vault.internal.company.com
      port: 8200
      authentication: KUBERNETES
      kubernetes:
        role: spring-app
      kv:
        enabled: true
        backend: secret
        application-name: my-service
```

HTTPS Configuration

```
# application.yml — SSL configuration
server:
  port: 8443
  ssl:
    key-store: classpath:keystore.p12  # Or /etc/ssl/app.p12 in production
    key-store-password: ${SSL_KEYSTORE_PASSWORD}
    key-store-type: PKCS12
    key-alias: app-ssl
    enabled: true
  http2:
    enabled: true  # Enable HTTP/2 for performance

# Force HTTPS redirect in Spring Security:
http.requiresChannel(channel -> channel.anyRequest().requiresSecure())
```

⚡ Quick Revision Notes

- Never commit secrets to git — use environment variables or Vault
- Use PKCS12 keystore for SSL (more portable than JKS)
- Let Nginx/load balancer handle SSL termination for better performance
- Rotate secrets regularly — 90 days for API keys, 1 year for SSL certs

Section 14 Testing Security

🧪 Testing Security

Unit Testing Secured Endpoints

```
@WebMvcTest(UserController.class)
@Import(SecurityConfig.class)
class UserControllerTest {

    @Autowired MockMvc mockMvc;

    // Test unauthenticated access → 401
    @Test
    void whenNoAuth_thenUnauthorized() throws Exception {
        mockMvc.perform(get("/api/users"))
            .andExpect(status().isUnauthorized());
    }

    // Test with mock user
    @Test
    @WithMockUser(roles = "USER")
    void whenUserRole_thenForbiddenForAdminEndpoint() throws Exception {
        mockMvc.perform(get("/api/admin/users"))
            .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void whenAdminRole_thenAllowed() throws Exception {
        mockMvc.perform(get("/api/admin/users"))
            .andExpect(status().isOk());
    }

    // Test with JWT token
    @Test
    void whenValidJwt_thenAllowed() throws Exception {
        String token = generateTestJwt("user@test.com", "ROLE_USER");
        mockMvc.perform(get("/api/users/profile")
                .header("Authorization", "Bearer " + token))
            .andExpect(status().isOk());
    }
}
```

```
// Custom security annotation for tests
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@WithSecurityContext(factory = WithMockJwtFactory.class)
public @interface WithMockJwt {
    String username() default "testuser";
    String[] roles() default {"ROLE_USER"};
    String[] scopes() default {};
}

@Component
public class WithMockJwtFactory implements WithSecurityContextFactory<WithMockJwt>
{
    @Override
    public SecurityContext createSecurityContext(WithMockJwt annotation) {
```

```
        // Create JWT authentication token for testing
        Map<String, Object> claims = new HashMap<>();
        claims.put("sub", annotation.username());
        claims.put("roles", Arrays.asList(annotation.roles()));
        Jwt jwt = Jwt.withTokenValue("test-token")
            .header("alg", "none")
            .claims(c -> c.putAll(claims))
            .issuedAt(Instant.now())
            .expiresAt(Instant.now().plusSeconds(3600))
            .build();

        JwtAuthenticationToken auth = new JwtAuthenticationToken(jwt);
        SecurityContext context = SecurityContextHolder.createEmptyContext();
        context.setAuthentication(auth);
        return context;
    }
}
```

⚡ Quick Revision Notes

- @WithMockUser is simple but doesn't test actual JWT parsing — use WithSecurityContextFactory for realistic tests
- Test both happy path (200 OK) and security failures (401/403) for every endpoint
- Integration tests should use real JWT tokens generated by your JwtTokenProvider
- Mock external OAuth2 providers in tests using WireMock or Spring's MockServer

Section 15 Real-World Project Walkthrough

🏗️ Real-World Project: Production Auth System

Project Structure

```
auth-system/
├── auth-service/                 # Spring Authorization Server
│   ├── src/main/java/
│   │   ├── config/
│   │   │   ├── AuthorizationServerConfig.java
│   │   │   └── SecurityConfig.java
│   │   ├── controller/
│   │   │   └── AuthController.java
│   │   ├── service/
│   │   │   ├── UserDetailsServiceImpl.java
│   │   │   └── RefreshTokenService.java
│   │   └── model/
│   │       ├── User.java, Role.java, Permission.java
│   │       └── RefreshToken.java
│   └── src/main/resources/
│       └── application.yml
│
├── api-gateway/                  # Spring Cloud Gateway
│   ├── src/main/java/
│   │   ├── config/GatewaySecurityConfig.java
│   │   └── filter/UserContextFilter.java
│   └── application.yml
│
├── user-service/                 # Resource Server
│   ├── src/main/java/
│   │   ├── config/ResourceServerConfig.java
│   │   └── controller/UserController.java
│   └── application.yml
│
├── product-service/              # Resource Server
│   └── ...
│
├── docker-compose.yml            # Full stack setup
└── k8s/                          # Kubernetes manifests
    ├── auth-service-deployment.yaml
    └── secrets.yaml
```

docker-compose.yml

```yaml
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: authdb
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: ["postgres_data:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
```

```
  auth-service:
    build: ./auth-service
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/authdb
      SPRING_DATASOURCE_USERNAME: ${DB_USER}
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
      SPRING_REDIS_HOST: redis
      APP_JWT_SECRET: ${JWT_SECRET}
    depends_on: [postgres, redis]
    ports: ["8080:8080"]

  api-gateway:
    build: ./api-gateway
    environment:
      AUTH_SERVICE_URL: http://auth-service:8080
    ports: ["80:8080"]
    depends_on: [auth-service]

volumes:
  postgres_data:
```

End-to-End API Flow Examples

1. Register

```
# POST /api/auth/register
curl -X POST http://localhost/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user@example.com",
    "password": "SecurePass123!",
    "firstName": "John",
    "lastName": "Doe"
  }'

# Response: 201 Created
{
  "id": "uuid-123",
  "email": "user@example.com",
  "message": "Registration successful. Please verify your email."
}
```

2. Login

```
# POST /api/auth/login
curl -X POST http://localhost/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"SecurePass123!"}'

# Response: 200 OK
{
  "accessToken": "eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1c2VyQGV4YW1wbGUuY29tIiwi...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "refreshToken": "set in HttpOnly cookie"
}
```

3. Access Protected API

```
# GET /api/users/profile (requires authentication)
curl -X GET http://localhost/api/users/profile \
  -H "Authorization: Bearer eyJhbGciOiJSUzI1NiJ9..."

# Response: 200 OK
{
  "id": "uuid-123",
  "email": "user@example.com",
  "roles": ["ROLE_USER"],
  "profile": {...}
}

# Without token → 401 Unauthorized
# With wrong role → 403 Forbidden
```

4. Refresh Access Token

```
# POST /api/auth/refresh (refresh token sent via cookie automatically)
curl -X POST http://localhost/api/auth/refresh \
  --cookie "refreshToken=<refresh-token-value>"

# Response: 200 OK
{
  "accessToken": "eyJhbGciOiJSUzI1NiJ9.NEW_TOKEN...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

5. OAuth2 Login

```
# 1. Redirect user to:
GET http://localhost/oauth2/authorization/google

# 2. After Google auth, callback:
GET http://localhost/login/oauth2/code/google?code=AUTH_CODE&state=STATE

# 3. Server exchanges code, creates/updates user, issues JWT
# 4. Redirect to frontend:
http://frontend.app/callback?token=JWT_ACCESS_TOKEN
```

⚡ Quick Revision Notes

- Keep auth-service, api-gateway, and business services completely separated
- Use docker-compose for local dev, Kubernetes for production
- Always test end-to-end flow including OAuth2, refresh token rotation, and role-based access

Section 16 Interview Preparation

❓ Interview Preparation — 50+ Questions

Core Spring Security Questions

Q: What is the difference between Authentication and Authorization?

A: Authentication verifies WHO you are (e.g., validating username/password, returns 401 on failure). Authorization determines WHAT you can do (e.g., role-based resource access, returns 403 on failure). Spring processes authentication first via AuthenticationManager, then authorization via AccessDecisionManager.

Q: How does Spring Security's filter chain work?

A: Spring Security installs a DelegatingFilterProxy in the Servlet filter chain. It delegates to FilterChainProxy which holds multiple SecurityFilterChain instances. Each request goes through the matching chain's ordered filters. Key filters: SecurityContextPersistenceFilter (loads context), UsernamePasswordAuthenticationFilter (handles form login), ExceptionTranslationFilter (converts exceptions to HTTP responses), FilterSecurityInterceptor (final authorization check).

Q: How is JWT validated in Spring Security?

A: 1) Extract Bearer token from Authorization header. 2) Parse JWT using JJWT/Nimbus. 3) Verify signature using the signing key. 4) Check expiration (exp claim). 5) Load UserDetails from DB using subject claim. 6) Create UsernamePasswordAuthenticationToken and set in SecurityContextHolder. If any step fails, return 401.

Q: What is the difference between HS256 and RS256 JWT algorithms?

A: HS256 (HMAC-SHA256) uses a shared secret — both parties must have the same key. RS256 (RSA-SHA256) uses asymmetric keys — private key signs, public key verifies. RS256 is preferred for microservices because only the auth server needs the private key; resource servers can freely get the public key via JWKS endpoint without security risk.

Q: How do you implement refresh token rotation?

A: On every refresh token use: 1) Validate the incoming refresh token. 2) Issue a new access token AND a new refresh token. 3) Immediately invalidate/delete the old refresh token. 4) If an already-used refresh token is presented again, it indicates theft — revoke ALL tokens for that user and force re-login.

Q: What is PKCE and when do you use it?

A: PKCE (Proof Key for Code Exchange) is an OAuth2 extension for public clients (SPAs, mobile apps) that can't safely store client secrets. The client generates a code_verifier (random string), creates code_challenge = BASE64URL(SHA256(verifier)), sends challenge in /authorize request, then sends verifier (not challenge) when exchanging the auth code for tokens. The server verifies SHA256(verifier) == challenge, preventing auth code interception attacks.

Q: How do you secure microservices with OAuth2?

A: Pattern: API Gateway validates JWTs, propagates user info (X-User-Id, X-User-Roles headers) to downstream services. Services trust gateway headers (internal network). Service-to-service communication uses Client Credentials grant (no user context). Shared Redis for token blacklist. RS256 JWT so resource servers can verify with public JWKS endpoint without calling auth server.

Q: What is the difference between @PreAuthorize and @Secured?

A: @PreAuthorize supports SpEL expressions (e.g., hasRole('ADMIN') and #id == authentication.principal.id). @Secured only supports role names (e.g., @Secured('ROLE_ADMIN')) — no expressions. @PreAuthorize is always preferred. Both require @EnableMethodSecurity annotation. @PostAuthorize checks after method execution, useful for filtering response data.

Q: How do you prevent CSRF attacks in Spring Boot?

A: For stateless REST APIs (JWT-based): disable CSRF (.csrf(csrf -> csrf.disable())) because CSRF exploits cookies/sessions which stateless APIs don't use. For session-based apps: keep CSRF enabled with CookieCsrfTokenRepository. The CSRF token must be included in state-changing requests (POST/PUT/DELETE). SPA apps read the CSRF cookie and include it as a header.

Q: What is SecurityContextHolder and what are its modes?

A: SecurityContextHolder stores the SecurityContext (which holds the Authentication) for the current thread. Modes: MODE_THREADLOCAL (default — each thread has its own context, cleared after request), MODE_INHERITABLETHREADLOCAL (child threads inherit parent's context — for async), MODE_GLOBAL (shared across all threads — for standalone apps only).

Q: How do you implement account lockout after failed login attempts?

A: Track failed attempts in Redis with TTL (e.g., redis.incr('lockout:' + username)). On N failures (e.g., 5), set user.accountLocked = true and store lockout expiry. In UserDetailsService, check user.isAccountNonLocked() and throw LockedException. Schedule auto-unlock after 30 minutes. Publish ApplicationEvent for audit logging.

Q: How do you handle JWT token revocation?

A: JWTs are stateless and self-contained — you can't truly invalidate them without state. Strategies: 1) Short expiry (15 min) — minimizes damage window. 2) Token blacklist in Redis — store jti claim of revoked tokens with TTL matching token expiry. 3) Refresh token revocation in DB — block token refresh. Best practice: Use all three — short access token, jti blacklist for critical revocations, DB refresh token control.

Q: What happens if SecurityContextHolder is not cleared?

A: If SecurityContextHolder.clearContext() is not called after a request, the ThreadLocal retains the Authentication object. In thread pool environments (Tomcat, Undertow), this thread is reused for a subsequent request — which could then have access to the previous user's SecurityContext. This is a serious security vulnerability. Spring Security's SecurityContextPersistenceFilter automatically clears the context after each request.

Q: How do you implement role hierarchy in Spring Security?

A: Define RoleHierarchy bean with setHierarchy('ROLE_ADMIN > ROLE_MANAGER > ROLE_USER'). This means ADMIN automatically has all MANAGER and USER permissions. Register it with DefaultWebSecurityExpressionHandler and set on SecurityFilterChain. Now hasRole('MANAGER') returns true for users with ROLE_ADMIN.

Q: What is the difference between OAuth2 and OpenID Connect (OIDC)?

A: OAuth2 is an authorization framework — it tells you what a client can access, not who the user is. OpenID Connect (OIDC) is an identity layer built on top of OAuth2 — it adds an ID Token (JWT) that tells you WHO the user is. Use OAuth2 for API access delegation, use OIDC for 'Login with Google/GitHub'. Spring Security supports both with spring-security-oauth2-client.

Scenario-Based Questions

Q: Your JWT tokens are being stolen and used from different IPs. How do you detect and respond?

A: 1) Include original client IP in JWT claims or bind token to IP hash. 2) Validate IP on every request (with allowance for mobile/VPN IPs). 3) Detect anomalies: token used from drastically different geo/IP → flag as suspicious. 4) Implement short-lived access tokens (15 min) to limit damage window. 5) Use token fingerprinting — include User-Agent hash in token. 6) Alert user via email when anomaly detected. 7) Force re-authentication from new location.

Q: How would you design auth for a system handling millions of requests per second?

A: 1) RS256 asymmetric JWT — resource servers verify locally without calling auth server (huge scalability benefit). 2) Cache public JWKS keys in memory (refresh every hour). 3) Redis cluster for refresh tokens and token blacklist. 4) Rate limiting at API Gateway level (Bucket4j or NGINX rate limiting). 5) Horizontal scaling of stateless auth service. 6) CDN for public endpoints. 7) Connection pooling for DB. Target: JWT validation is in-memory = microseconds per request.

Q: A user reports they were logged in as another user. What could cause this and how do you investigate?

A: Root causes: 1) SecurityContextHolder not cleared between requests (ThreadLocal leak in thread pool). 2) Session fixation attack. 3) Race condition in session management. Investigation: 1) Check if SecurityContextPersistenceFilter is properly configured. 2) Review session management settings. 3) Check for concurrent modification of SecurityContext. 4) Audit logs for the affected user. Fix: Enable strict session management, ensure SecurityContextHolder.clearContext() is called, use STATELESS policy for REST APIs.

Q: How would you implement multi-factor authentication (MFA) in Spring Security?

A: 1) After first-factor (password) authentication, don't set SecurityContext yet. 2) Issue a short-lived (5 min) 'mfa-pending' JWT with limited scope. 3) Client sends this token + TOTP code/SMS OTP to /auth/mfa/verify. 4) Verify OTP (use Google Authenticator TOTP library or Twilio for SMS). 5) If valid, issue full JWT. Implementation: Custom AuthenticationProvider that checks MFA status, custom filter for MFA verification endpoint, store MFA secret encrypted in user table.

Common Interview Traps

Common Mistake	Correct Answer
'JWT is encrypted'	WRONG — JWT payload is Base64url ENCODED (not encrypted). Anyone can decode it. Use JWE for encryption.
'Disable CSRF for all endpoints'	Only safe for stateless REST APIs. Session-based apps MUST keep CSRF enabled.
'Store JWT in localStorage'	Vulnerable to XSS attacks. Store access token in memory, refresh token in HttpOnly cookie.
'hasRole() and hasAuthority() are the same'	hasRole('ADMIN') auto-adds 'ROLE_' prefix. hasAuthority('ADMIN') does not. Use hasAuthority('ROLE_ADMIN') for exact match.
'OAuth2 is for authentication'	OAuth2 is for AUTHORIZATION. OIDC (built on OAuth2) adds authentication via ID Token.
'Long-lived tokens are fine'	Access tokens should expire in 15 min. Long-lived tokens = larger attack window if compromised.

⚡ Quick Revision Notes

- Authentication = 401, Authorization = 403 — never confuse these
- JWT = Base64url encoded, NOT encrypted — never put sensitive data in payload
- RS256 for microservices (public key JWKS), HS256 for single-service (shared secret)
- PKCE = for public clients (SPA/mobile) that can't store secrets
- OIDC = OAuth2 + ID Token — for 'Login with Google', not just API access
- Refresh token rotation = issue new + invalidate old on every use

Section 17 Anti-Patterns

⚠️ Common Mistakes & Anti-Patterns

Anti-Pattern	Correct Approach
JWT in localStorage	XSS attack can steal token — use memory (JS) + HttpOnly cookie for refresh token
Weak JWT secret	Short secrets can be brute-forced — use 256-bit+ random secret (openssl rand -base64 32)
No token expiry	JWT without exp claim is valid forever if signing key not rotated — always set exp
Logging tokens/passwords	Credentials in logs = security incident — log user IDs only, never credentials
permitAll() for sensitive endpoints	Accidentally exposing /actuator, /h2-console, /swagger — explicitly secure or disable in production
Storing plain passwords	NEVER store plain text or MD5/SHA1 hashed passwords — always BCrypt/Argon2
CORS wildcards	Access-Control-Allow-Origin: * with credentials allowed = allows any site to read your API
Trusting user-provided roles	Never accept roles from JWT payload claims without server-side validation
Sharing signing keys	Using same JWT signing key across environments — rotate keys per environment
No rate limiting	Brute force login attempts succeed without rate limiting and account lockout

Production Security Checklist

```
✅ AUTHENTICATION
☐ BCrypt/Argon2 password hashing (strength >= 10)
☐ Account lockout after 5 failed attempts
☐ Email verification for new accounts
☐ Secure password reset flow (time-limited tokens)
☐ MFA support (TOTP/SMS) for admin accounts

✅ JWT / TOKENS
☐ Short access token expiry (15 minutes)
☐ Refresh token rotation on every use
☐ Refresh tokens stored in DB/Redis (not in JWT)
☐ Token revocation via Redis blacklist
☐ RS256 algorithm for microservices
☐ jti claim included for revocation capability
☐ No sensitive data in JWT payload

✅ API SECURITY
☐ HTTPS everywhere (TLS 1.2+)
☐ HSTS header with preload
```

```
☐ Content Security Policy headers
☐ Rate limiting (per IP + per user)
☐ Input validation on all endpoints
☐ CORS restricted to known origins only

✅ OAUTH2
☐ PKCE for all public clients
☐ State parameter validated to prevent CSRF
☐ Short-lived authorization codes (10 min)
☐ Scope minimization (request only needed scopes)

✅ MONITORING
☐ Security events logged (login success/failure, access denied)
☐ Alerts on brute force detection
☐ Token theft detection (anomalous IP/device)
☐ Audit log for admin actions

✅ DEPLOYMENT
☐ Secrets in Vault or environment variables (not code)
☐ Different keys per environment (dev/staging/prod)
☐ Dependency vulnerability scanning (Snyk/OWASP)
☐ Penetration testing before production launch
```

⚡ Quick Revision Notes

- Production checklist = your security PR review checklist — use it before every release
- Most breaches happen due to misconfigurations, not zero-days — follow the checklist
- Security debt compounds — fix issues early before they become breach vectors

Section 18 How to Think About Security

🧠 How to Think About Security Design

JWT vs OAuth2 — Decision Framework

Scenario	Recommendation
Use JWT when...	Single microservice or monolith, Internal API auth, You control both client and server, Simple role-based access
Use OAuth2 when...	Users need to grant third-party app access to their data, Multiple resource servers need token validation, Social login required (Google/GitHub), Fine-grained scope-based authorization needed
Use OIDC when...	You need to verify USER IDENTITY (not just API access), Implementing 'Login with...' functionality, SSO across multiple applications
Use Both (JWT + OAuth2) when...	Microservices architecture with OAuth2 for service auth AND custom JWT claims for RBAC

Security Design Interview Framework

When asked to design a secure system in an interview, follow this structure:

1. Define the threat model — who are the attackers, what are the assets, what are attack vectors?
2. Choose authentication mechanism — stateless JWT for APIs, OAuth2 for delegated access, OIDC for identity
3. Design authorization model — RBAC vs ABAC, granularity of permissions, role hierarchy
4. Plan token lifecycle — expiry, rotation, revocation strategy
5. Consider scalability — stateless auth scales horizontally, Redis for shared state
6. Add monitoring — security events, anomaly detection, audit trails
7. Plan for failure — what happens if auth service is down? Token validation cached?

Security Mindset Principles

🔐 **Principle of Least Privilege**

Every component should have ONLY the permissions it needs. A read-only service should not have write tokens. A user service should not access payment data. Apply this to: JWT scopes, DB user permissions, API access, microservice-to-service calls.

🛡️ Defense in Depth

Multiple layers of security. If JWT validation fails, authorization check should still work. If authorization fails, audit logging should still capture it. If audit fails, rate limiting should still protect. Never rely on a single security control.

🚫 Zero Trust Architecture

Never trust, always verify. Even internal microservices should authenticate each other (mTLS or service account tokens). Assume breach — design systems to limit blast radius when one service is compromised.

💡 **Pro Tip:** In system design interviews, always bring up security proactively. Say: 'For authentication, I'd use JWT with 15-minute expiry and refresh token rotation. For authorization, RBAC with role hierarchy. For service-to-service, Client Credentials OAuth2. And RS256 so each service can verify tokens without calling the auth server.' This signals senior-level thinking.

⚡ Quick Revision Notes

- JWT = stateless token for API auth. OAuth2 = delegated authorization framework. OIDC = identity layer on OAuth2
- Decision: Simple internal API → JWT. Third-party access → OAuth2. Social login → OIDC
- Threat model first: identify assets (user data, tokens), actors (external, internal, compromised service), and mitigations
- Least privilege: grant minimum permissions needed, at every layer (API, DB, service-to-service)
- Zero trust: verify everything, even internal calls — mTLS for service mesh in production

— End of Spring Security Mastery Guide —